Experiment 5

Hands-on Solidity programming assignments for creating smart contracts

Aim

To gain hands-on experience with core Solidity programming constructs — the same ground covered by Remix IDE's LearnETH "Solidity Basics" course (19 guided assignments) — by writing, compiling, deploying, and exercising a single teaching contract that demonstrates each construct with a real, executed transaction on a local blockchain.

Theory

Remix IDE's "Solidity Basics" LearnETH course walks a learner through Solidity's fundamentals one guided assignment at a time: declaring state variables of different types, writing functions with different visibility and mutability modifiers, control-flow constructs, structs and mappings for structured/keyed storage, arrays, events for on-chain logging, and modifiers for reusable access-control checks. Rather than stepping through the course's 19 separate assignments in the Remix browser UI, this experiment consolidates the same set of constructs into one contract, `SolidityBasics.sol`, deployed and interacted with via a scripted local Hardhat node (the same substitution used for the other Remix/Metamask-based experiments in this course):

- **State variables & data types** — `uint256`, `int256`, `bool`, `address`, `string`, and `bytes32` are all declared as public state variables, each auto-generating a getter.
- **Structs & arrays** — a `Student { name, age, marks }` struct is stored both in a dynamic array (`Student[] public students`) and in a `mapping(uint256 => Student)` keyed by an auto-incrementing id, showing the two common ways Solidity stores structured records.
- **Mappings** — `mapping(address => uint256) public balances` tracks per-address ETH deposits made through a `payable` function.
- **Events** — `GreetingChanged`, `StudentAdded`, and `Deposited` are emitted on state changes and decoded back client-side via `ethers.js`'s `interface.parseLog`.

- **Modifiers & access control** — `onlyOwner` restricts `setGreeting` to the deploying address; a non-owner call is shown reverting.
- **Control structures** — `classify()` uses an if/else-if/else chain to grade a student's marks, `averageMarks()` uses a `for` loop to sum an array, and `isEven()` uses a ternary expression.
- **Function visibility** — `view`, `pure`, `external`, `public`, `internal`, and `private` are all represented; a `private` counter (`secretCounter`) has no public getter and is only reachable indirectly through a `public` wrapper function (`bumpCounter`), demonstrating that Solidity's `private` keeps a variable's value out of any public getter while still leaving it readable off-chain via storage slots — a distinction the LearnETH course specifically covers.

Tools used: Node.js 24, Hardhat 2.29 with `@nomicfoundation/hardhat-toolbox` (bundles `ethers.js` v6), Solidity 0.8.24. Run directly in a terminal against a standalone `npx hardhat node` (a second instance on port 8546, alongside Experiment 4's node on 8545), in place of the Remix IDE browser environment the LearnETH course normally runs in.

Output

1. Compile the SolidityBasics contract

```
$ npx hardhat compile
Compiled 1 Solidity file successfully (evm target: paris).
```

2. Start a second local Hardhat node (port 8546) and deploy

```
$ npx hardhat run scripts/deploy_and_interact.js --network localhost

=== Accounts (Hardhat local network) ===
Owner: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
Alice: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8

SolidityBasics deployed at: 0x5FbDB2315678afecb367f032d93F642f64180aa3
```

A fresh local chain, kept separate from Experiment 4's node so both experiments' deployments and gas accounting can be inspected independently.

3. State variables & data types

```
=== 1. State variables & data types ===
Initial greeting: Hello, Blockchain!
Initial fixedBytes: SOLIDITY
unsignedNumber = 42
```

```
signedNumber   = -17
flag after toggle: true
```

Confirms `uint256`, `int256`, `bool`, `string`, and `bytes32` state variables all read back correctly after being set through their respective setter functions.

4. Modifiers, events & access control

```
=== 2. Modifiers, events & access control ===
GreetingChanged event: "Hello, Blockchain!" -> "Hi from the owner!"
Negative test: non-owner (Alice) tries to change the greeting...
Reverted as expected: Error: VM Exception while processing transaction: reverted with reason string 'Not the
owner'
```

The `GreetingChanged` event is correctly emitted and decoded off-chain, and the `onlyOwner` modifier correctly blocks Alice (a non-owner) from calling `setGreeting`.

5. Payable functions & mappings

```
=== 3. Payable functions & mappings ===
Alice deposited 0.5 ETH (tx 0x3e33a644ca6677509b1581c23280badcd3bf0fe1ed497a93918afb655394e5f2)
balances[Alice] = 0.5 ETH
```

A `payable` function correctly accepts ETH and records it against the caller's address in the `balances` mapping.

6. Structs, arrays & control structures

```
=== 4. Structs, arrays & control structures ===
Added student Asha (age 20, marks 82)
Added student Ravi (age 21, marks 67)
Added student Meera (age 20, marks 45)
Added student Zoya (age 22, marks 30)
Total students: 4
Average marks: 56
Student #0: Asha, marks 82 -> classify() = "Distinction"
Student #1: Ravi, marks 67 -> classify() = "First Class"
Student #2: Meera, marks 45 -> classify() = "Pass"
Student #3: Zoya, marks 30 -> classify() = "Fail"
```

Four `Student` structs are pushed into a dynamic array; `averageMarks()`'s `for`-loop correctly computes the integer average (56), and `classify()`'s if/else-if chain correctly grades each student's marks into the right band.

7. Pure functions (arithmetic, ternary)

```
=== 5. Pure functions (arithmetic, ternary) ===
add(15, 27) = 42
isEven(10) = true
isEven(7)  = false
```

`pure` functions perform arithmetic and a ternary parity check with no state access, confirmed against known inputs.

8. Function visibility (internal via public wrapper)

```
=== 6. Function visibility (internal via public wrapper) ===
bumpCounter() call #1, tx 0x856c68727c...
bumpCounter() call #2, tx 0xe1f865d1e4...
bumpCounter() call #3, tx 0x3498034be3...
Note: secretCounter itself is private and has no public getter,
      it is only reachable indirectly through bumpCounter().
```

Three separate transactions call the `external` `bumpCounter()` wrapper, which is the only way to advance the `private` `secretCounter` state variable — demonstrating Solidity's visibility levels working together (`private` storage, an `internal` helper, and a `public` / `external` entry point).

(Full console log saved as `deploy_output.log`, and the Hardhat node's own server-side log as `hardhat_node.log`, both in this folder.)

Conclusion

This experiment covered the same ground as Remix IDE's LearnETH "Solidity Basics" course — state variables and data types, structs, mappings, arrays, events, modifiers, control structures, and the full range of function visibility/mutability keywords — by writing, compiling, and deploying a single consolidated `SolidityBasics` contract to a local Hardhat node and exercising every construct with a real, mined transaction rather than working through the course's assignments one at a time in the Remix browser UI. Each construct's behaviour was independently verified: the owner-only `setGreeting` modifier correctly blocked a non-owner caller, the `for`-loop-based `averageMarks()` and if/else-chain `classify()` produced correct results against four struct records, and the `private` `secretCounter` was shown to be mutable only through its `public` wrapper — confirming that Solidity's access-control and visibility guarantees hold identically whether driven from Remix + Metamask or from a scripted `ethers.js` client against a local chain.